

Chapter 18: B-Trees

1) B-trees are balanced search trees designed to work well on magnetic disks or other direct-access secondary storage devices. Many database systems use B-trees, or variants of B-trees, to store information.

B-tree nodes may have many children, from a handful to thousands. That is, the "branching factor" of a B-tree can be quite large, although it is usually determined by the characteristic of the disk unit used.

2) Computer memory is of two types:-

- i) primary memory comprising of silicon memory chips
- ii) secondary memory comprising of magnetic disks

A disk drive consists of several platters which rotate at a constant speed around a common spindle. The surface of each platter is covered with magnetic material. Each platter is read or written by a head at the end of an arm. The arms are geared together and they can move their heads towards or away from the spindle.

When a given head is stationary the surface that passes underneath it is called a track. The set of concentric tracks underneath the read-write heads is known as a cylinder.

Disks are much slower than main memory because they have moving parts. There are 2 components to mechanical motion: platter rotation and arm movement.

In order to amortize the time spent waiting for mechanical movement on disks read or write one or more pages at a time. We shall look at two principal components of running time:-

(i) the no. of disk access measured in terms of the no. of pages of information that need to be read from or written to the disk.

(ii) the CPU (computing) time.

→ In a typical B-tree application, the amount of data handled is so large, that the entire data cannot fit into main memory at once.

The B-tree algorithms copy selected pages from disk into main memory as needed and write back onto disk the pages that have changed. B-tree algorithms are designed so that only a constant number of pages are in main memory at any time. Thus, the size of main memory does not limit the size of B-trees that can be handled.

→ We model disk operations in pseudocode as follows: let x be a pointer to an object. If the object is currently in computer's main memory then we can refer to the fields of the object as usual, $\text{Key}[x]$ for example. However, if the object referred to by x resides on disk, we must perform the operation $\text{DISK-READ}(x)$ to read x to main memory before we can access its fields. If x is already in memory then the operation $\text{DISK-READ}(x)$ is a "no-op".

DISK-WRITE(x) is used to save any changes that have been made to the fields of object x . The typical pattern for working with an object is as follows:-

$x \leftarrow$ pointer to some object

DISK-READ(x)

operations that access/modify fields of x

DISK-WRITE(x) > omitted if no fields of x were changed

other operations that only access but do not modify the fields of x

→ The operating system can keep only a limited number of pages in main memory at any one time. Pages no longer in use are flushed from main memory by the operating system; the B-tree algorithms will ignore this issue.

→ The running time of a B-tree algorithm is determined mainly by the number of DISK-READ and DISK-WRITE operations it performs. Hence, it is sensible to use these operations efficiently by having them read or write as much information as possible. Thus, a B-tree node is usually as large as a whole disk page. The number of children a B-tree node can have is therefore limited by the size of a disk page.

→ For a large B-tree stored on a disk, branching factors between 50 and 2000 are often used, depending upon the size of a key relative to the size of a page. A large branching factor dramatically reduces both the height of the tree and the no. of disk accesses required to find any key.

B) Definition of B-trees:

To keep things simple we assume that any satellite information associated with a key is stored in the same node as the key. In practice one might actually store with a key, just a pointer to another disk page containing the satellite information for that key.

A B-tree is a rooted tree having the following properties:

- 1) Every node x has the following fields:-
 - Ⓐ the no. of keys currently stored in x i.e. $n[x]$
 - Ⓑ all the $n[x]$ keys stored in non-decreasing order: $key_1[x] \leq key_2[x] \leq key_3[x] \dots \leq key_{n[x]}[x]$
 - Ⓒ a boolean value $leaf[x]$ which is true if x is a leaf and false if x is an internal node
- 2) Every internal node x also contains $n[x]+1$ pointers to its children: $c_1[x], c_2[x], c_3[x], \dots, c_{n[x]+1}[x]$. leaf nodes have no children, so their c_i fields are undefined.

3) All the Keys stored in the subtree rooted at x are partitioned into $n[x]+1$ subranges by the Keys stored in x :

$$K_1 \leq \text{Key}_1[x] \leq K_2 \leq \text{Key}_2[x] \leq \dots \leq \text{Key}_{n[x]}[x] \leq K_{n[x]+1}$$

4) All leaves are at the same depth = height of the B-tree.

5) There are lower and upper bounds on the no. of Keys a node can contain. If t be the "minimum degree" of a B-tree then:-

Ⓐ $t \geq 2$

Ⓑ Every node other than the root must have at least $t-1$ Keys. Every internal node other than the root has at least t children. If the tree is non-empty the root has at least one Key.

Ⓒ Every node can contain at most $2t-1$ Keys. Therefore an internal node can have at most $2t$ children. If a node contains exactly $2t-1$ Keys, it is said to be full.

→ The simplest possible B-tree occurs when $t=2$ and is called a 2-3-4 tree. Every internal node in a 2-3-4 tree has either 2, 3 or 4 children.

4) Height of a B-tree:

The no. of disk accesses required for most operations on a B-tree is proportional to the height of the B-tree.

Theorem

Let n be the no. of keys stored in a B-tree of height h and minimum degree t , we have :-

$$h \leq \log_t \frac{n+1}{2}$$

Proof: The root has at least 1 key and all other nodes contain at least $t-1$ keys. Thus, there are at least 2 nodes at depth 1, at least $2t$ nodes at depth 2, at least $2t^2$ nodes at depth 3 and so forth, until at depth h there are $2t^{h-1}$ nodes.

$$\text{Thus, } \underset{\substack{\downarrow \\ \text{no. of keys}}}{n} \geq 1 + (t-1) \sum_{i=1}^h 2t^{i-1} = 1 + 2(t-1) \left(\frac{t^h - 1}{t-1} \right)$$

$$\Rightarrow n \geq 2t^h - 1$$

$$\Rightarrow t^h \leq \frac{(n+1)}{2} \Rightarrow h \leq \log_t \left(\frac{n+1}{2} \right)$$

→ Although the height of both B-trees and red-black trees grows as $O(\lg n)$ { where n = the no. of keys }, for B-trees the base of the logarithm can be many times larger. Thus B-trees save a factor of about $\lg t$ over red-black trees in the no. of nodes examined for most tree operations. Since examining a node usually requires a disk access, the no. of disk accesses is substantially reduced.

5) Basic operations on B-trees:

We adopt the convention that the root of the B-tree is always in main memory so that a DISK-READ on the root is never required. However, a DISK-WRITE of the root may be required whenever the root node is changed.

All the procedures that are presented are "one-pass" algorithms that proceed downward from the root of the tree, without having to back up.

Searching a B-tree:

Searching a B-tree is much like searching a binary search tree, except that instead of making a binary branching decision at each node, we make a multiway branching decision depending upon the number of children of a node.

The B-TREE-SEARCH procedure takes as input a pointer to the root of the subtree and a key to be searched for in that subtree. It returns an ordered pair (y, i) such that the i th key of a node y contains the input key. Otherwise NIL is returned.

B-TREE-SEARCH(x, k)

```
1   $i \leftarrow 1$ 
2  While  $i \leq n[x]$  and  $k > \text{Key}_i[x]$ 
3      do  $i \leftarrow i+1$ 
4  If  $i \leq n[x]$  and  $k = \text{Key}_i[x]$ 
      then return  $(x, i)$ 
5  If leaf  $[x]$ 
```

```

6      then return NIL
7      else DISK-READ (  $U[x]$  )
8      return B-TREE-SEARCH (  $U[x]$  ,  $k$  )

```

→ The nodes encountered during recursion form a path downward from the root of the tree. The no. of disk pages accessed by B-TREE-SEARCH is therefore $O(h) = O(\log_t n)$.

Since $n[x] \leq 2t-1$ the max time of the while loop is $O(t)$ and since the procedure can recurse a maximum of h times, the total CPU time is $O(ht) = O(t \log_t n)$.

6) Creating an empty B-tree:

To build a B-tree we first use the procedure B-TREE-CREATE to create an empty root node and then call B-TREE-INSERT to add new keys. The procedure ALLOCATE-NODE allocates one disk page to be used as a new node in $O(1)$ time.

B-TREE-CREATE(T)

```

1   $x \leftarrow \text{ALLOCATE-NODE}()$ 
2   $\text{leaf}[x] \leftarrow \text{TRUE}$ 
3   $n[x] \leftarrow 0$ 
4  DISK-WRITE( $x$ )
5   $\text{root}[T] \leftarrow x$ 

```

→ B-TREE-CREATE requires $O(1)$ disk page and $O(1)$ CPU time.

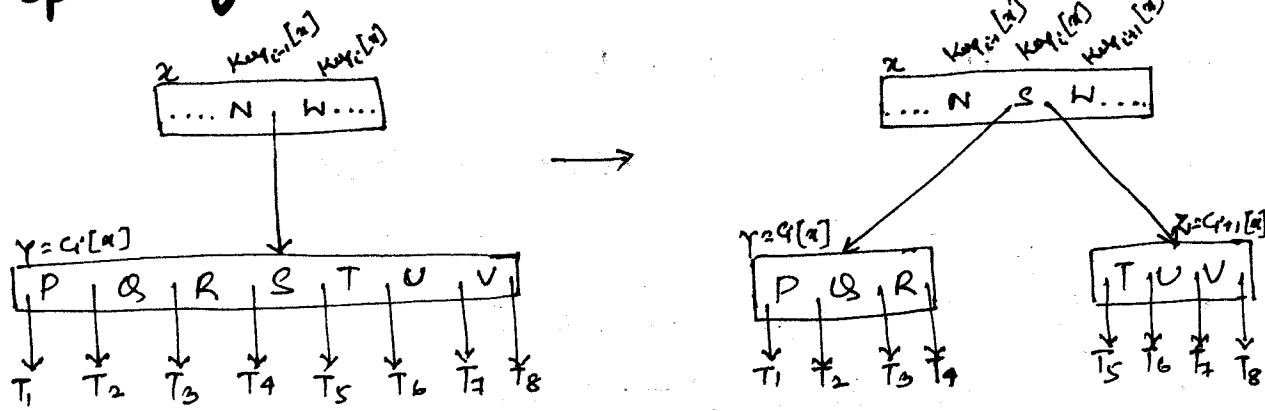
7) Inserting a Key into a B-tree:

We search for a leaf position at which to insert the new key. We cannot simply create a new leaf node and insert it, as the resulting tree would fail to be a valid B-tree. Instead, we need to insert the new key into an existing leaf node.

Since we cannot insert a key into a leaf node that is full, we introduce an operation that splits a full node y (having $2t-1$ keys) around its median key $key_t[y]$ into 2 nodes having $t-1$ keys each. The median key moves up into y 's parent to identify the dividing point between the 2 new trees. But if y 's parent is also full, it must also be split before the new key can be inserted, and thus this need to split full nodes can propagate all the way up to the tree.

We can insert a key into a B-tree in a single pass down the tree from the root to a leaf. To do so, we do not wait to find out whether we will actually need to split a full node in order to do the insertion. Instead, as we travel down the tree searching for the appropriate position to insert the new key, we split each full node that we come across. Thus, when we want to split a full node, we are assured that its parent is not full.

splitting a node in a B-tree:



The procedure B-TREE-SPLIT-CHILD splits the i 'th child y of a "non-full" node x , i.e. $y = C_i(x)$ is a full child. x and y are assumed to be in main memory (It is assumed that any node passed as a parameter to some procedure has a DISK-READ performed on it before the function call).

To split a full root, we first create a new empty node and make the root the only child of it so that we can use B-TREE-SPLIT-CHILD. A B-tree grows in height only by splitting a root. Splitting any other node of the B-tree adds a new node at the same depth as the node to be split and hence does not affect the height of the B-tree.

B-TREE-SPLIT-CHILD(x, i, y)

- 1 $z \leftarrow \text{ALLOCATE-NODE}()$
- 2 $\text{leaf}[z] \leftarrow \text{leaf}[y]$
- 3 $n[z] \leftarrow t-1$
- 4 for $j \leftarrow 1$ to $t-1$
- 5 do $\text{key}_j[z] \leftarrow \text{key}_{t+j}[y]$

of the subtree being operated upon is a leaf or not. If it's a leaf the new key is inserted into the root else you repeat the procedure on an appropriate subtree. Before the recursion descends to a subtree, you first check to see if the root of the subtree is full. If it is full, that root is first split so that B-TREE INSERT-NONFULL can be invoked on it.

B-TREE-INSERT-NONFULL (root, K)

```

1  i ← n[root]
2  if leaf[root]
3      then while i ≥ 1 and K < Keyi[root]
4              do Keyi+1[root] ← Keyi[root]
5              i ← i - 1
6  Keyi+1[root] ← K
7  n[x] ← n[x] + 1
8  DISK-WRITE(x)
9  else while i ≥ 1 and K < Keyi[x]
10         do i ← i - 1
11         i ← i + 1 ▷ recursion descends to the i+1 th subtree
12         DISK-READ (x[i])
13         if n[x[i]] = 2t - 1
14             then B-TREE-SPLIT-CHILD (x, i, x[i])

```

▷ Because you have split the ith child of root, the new key has 2 subtrees ~~as~~ as its new options :- the old ith child of x (which is

reinserts the new i th child) and the new $(i+1)$ th child

15 If $K > \text{Key}_i[x]$ $\triangleright$ these are separated by the i th key of x

16 then $i \leftarrow i+1$

17 B-TREE-INSERT-NONFULL($C_i[\text{root}], K$)

$\rightarrow$ The number of disk accesses performed by B-TREE-INSERT is $O(h)$ for a B-tree of height h , since only $O(1)$ DISK-READ and DISK-WRITE operations are performed between calls to B-TREE-INSERT-NONFULL. The total CPU time used is $O(th) = O(t \log n)$.

Since B-TREE-INSERT-NONFULL is tail-recursive, it can be alternately implemented as a while loop, demonstrating that the no. of pages that need to be in main memory at any time is $O(1)$.

$\rightarrow$ To insert a key K into any B-tree T we invoke the procedure B-TREE-INSERT(T, K):

B-TREE-INSERT(T, K)

$\triangleright$ B-TREE-INSERT uses the auxiliary procedure B-TREE-INSERT-NONFULL to insert key K into a tree rooted at a non-full root. If the root were full it first splits the root so that B-TREE-INSERT-NONFULL can be invoked

```

1   $r \leftarrow \text{root}[T]$ 
2  if  $n[r] = 2t - 1$ 
3      then  $s \leftarrow \text{ALLOCATE\_NODE}()$ 
4            $\text{root}[T] \leftarrow s$ 
5            $\text{leaf}[s] \leftarrow \text{false}$ 
6            $n[s] \leftarrow 0$ 
7            $G[s] \leftarrow r$ 
8            $\text{B-TREE\_SPLIT\_CHILD}(s, 1, r)$ 
9            $\text{B-TREE\_INSERT\_NONFULL}(s, K)$ 
10      else
11           $\text{B-TREE\_INSERT\_NONFULL}(r, K)$ 

```

B-TREE INSERT requires $O(h)$ disk accesses and $O(h)$
 $= O(t \log_t n)$ CPU time.